Milestone 2: Data Exploration Project: NYC Property Tax Valuation Classification

This notebook explores a merged dataset of New York City property sales and assessment records from 2022 to 2024. Our project aims to classify properties as undervalued, fairly valued, or overvalued by comparing assessed values with observed sale prices and using geographic, tax-related, and structural property features. The merged dataset is appropriate for this question because it combines both transaction-level sales information and official assessment information in one file.

In [26]:
```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_parquet("../data/merged_2022_2024.parquet")
print(df.shape)
df.head()
```

```
(161045, 48)
```

Out[26]:

| | BOROUGH | NEIGHBORHOOD | BUILDING CLASS CATEGORY | BLOCK_sale | LOT_sale | ADDRE |
|---|---|---|---|---|---|---|
| **0** | 2 | BATHGATE | 01 ONE FAMILY DWELLINGS | 2905 | 26 | 16 WASHINGT AVEN |
| **1** | 2 | BATHGATE | 01 ONE FAMILY DWELLINGS | 3028 | 24 | 410 EA 179 STRE |
| **2** | 2 | BATHGATE | 01 ONE FAMILY DWELLINGS | 3030 | 65 | 4455 PA AVEN |
| **3** | 2 | BATHGATE | 01 ONE FAMILY DWELLINGS | 3037 | 101 | 443 EAST STRE |
| **4** | 2 | BATHGATE | 01 ONE FAMILY DWELLINGS | 3039 | 29 | 23 WASHINGT A |

5 rows × 48 columns

In [27]:
```
import numpy as np

df["target_ratio"] = df["TAX CLASS AT TIME OF SALE"].astype(str).apply
    lambda x: 0.06 if x.strip().startswith("1") else 0.45
)

df["ratio_vs_target"] = df["assessment_ratio"] / df["target_ratio"]
```

```
df["label"] = pd.cut(
    df["ratio_vs_target"],
    bins=[0, 0.90, 1.10, float("inf")],
    labels=["undervalued", "fairly_valued", "overvalued"]
)

df = df.dropna(subset=["label"])
```

In [28]:
```
df[["TAX CLASS AT TIME OF SALE", "assessment_ratio", "target_ratio", "
```

Out[28]:

| | TAX CLASS AT TIME OF SALE | assessment_ratio | target_ratio | ratio_vs_target | label |
|---|---|---|---|---|---|
| **0** | 1.0 | 0.041143 | 0.06 | 0.685714 | undervalued |
| **1** | 1.0 | 0.051206 | 0.06 | 0.853435 | undervalued |
| **2** | 1.0 | 0.062538 | 0.06 | 1.042308 | fairly_valued |
| **3** | 1.0 | 0.018333 | 0.06 | 0.305556 | undervalued |
| **4** | 1.0 | 0.047520 | 0.06 | 0.792002 | undervalued |
| **5** | 1.0 | 0.061903 | 0.06 | 1.031720 | fairly_valued |
| **6** | 1.0 | 0.067667 | 0.06 | 1.127778 | overvalued |
| **7** | 1.0 | 0.061900 | 0.06 | 1.031667 | fairly_valued |
| **8** | 1.0 | 0.001934 | 0.06 | 0.032235 | undervalued |
| **9** | 1.0 | 0.060000 | 0.06 | 1.000000 | fairly_valued |

In [29]:
```
df["label"].value_counts(dropna=False)
```

Out[29]:
```
label
undervalued      130374
fairly_valued     15995
overvalued        14676
Name: count, dtype: int64
```

In [30]:
```
df.columns.tolist()
```

Out[30]:
```
['BOROUGH',
 'NEIGHBORHOOD',
 'BUILDING CLASS CATEGORY',
 'BLOCK_sale',
 'LOT_sale',
 'ADDRESS',
 'APARTMENT NUMBER',
 'ZIP CODE',
 'RESIDENTIAL UNITS',
 'COMMERCIAL UNITS',
 'TOTAL UNITS',
 'LAND SQUARE FEET',
 'GROSS SQUARE FEET',
 'YEAR BUILT',
 'TAX CLASS AT TIME OF SALE',
 'BUILDING CLASS AT TIME OF SALE',
 'SALE PRICE',
 'SALE DATE',
 'SOURCE_FILE',
 'SALE_YEAR',
 'BBL',
 'BORO',
 'BLOCK_assess',
 'LOT_assess',
 'BLDG_CLASS',
 'FINACTTOT',
 'FINACTLAND',
 'FINMKTTOT',
 'PYACTTOT',
 'PYACTLAND',
 'GROSS_SQFT',
 'LAND_AREA',
 'NUM_BLDGS',
 'YRBUILT',
 'BLD_STORY',
 'UNITS',
 'COOP_APTS',
 'LOT_FRT',
 'LOT_DEP',
 'LOT_IRREG',
 'STREET_NAME',
 'ZIP_CODE',
 'ZONING',
 'FISCAL_YEAR',
 'TAX_YEAR',
 'FINACTTOT_per_unit',
 'assessment_ratio',
 'label',
 'target_ratio',
 'ratio_vs_target']
```

In [31]:
```
df.isnull().sum().sort_values(ascending=False).head(20)
```

Out[31]:
```
APARTMENT NUMBER     119446
LAND SQUARE FEET      86387
GROSS SQUARE FEET     86387
COMMERCIAL UNITS      81224
RESIDENTIAL UNITS     49707
TOTAL UNITS           44544
YEAR BUILT            10556
ZIP CODE                  8
LOT_FRT                   1
NUM_BLDGS                 1
ZONING                    1
BLD_STORY                 1
UNITS                     0
COOP_APTS                 0
BOROUGH                   0
LOT_IRREG                 0
YRBUILT                   0
LAND_AREA                 0
LOT_DEP                   0
TAX_YEAR                  0
dtype: int64
```

Dataset overview and data quality

The merged dataset contains 161,045 observations and 48 variables, which is far above the minimum sample size required for the project. The variables include borough, neighborhood, zip code, zoning, tax class, building characteristics, assessed value per unit, assessment ratio, and the final label used for classification.

The missing-value analysis shows that some structural variables are incomplete. In particular, apartment number, gross square feet, land square feet, and unit counts have substantial missingness. This is important because it suggests that some features may need imputation, transformation, or careful selection before model training. At the same time, the key variables needed for the core classification problem, such as assessed value information and the final label, appear to be largely available.

In [32]:
```
df["label"].value_counts()
```

Out[32]:
```
label
undervalued      130374
fairly_valued     15995
overvalued        14676
Name: count, dtype: int64
```

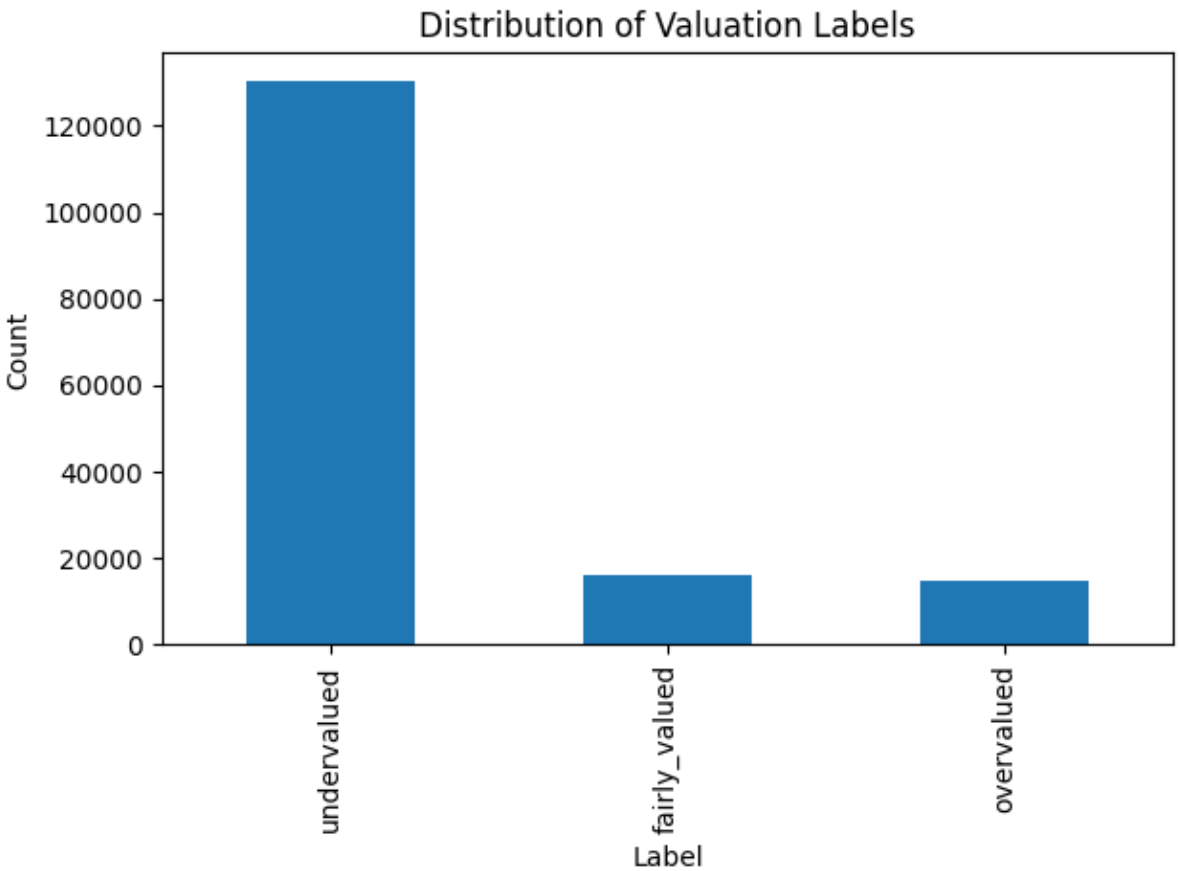
Target variable and assessment ratio

A central part of the project is the target label that classifies each property as

undervalued, fairly valued, or overvalued. In the modeling pipeline, this label is based on how each property's assessment ratio compares with a tax-class-specific target ratio. For tax class 1 properties, the target ratio is set to 0.06, while for the other tax classes in our sample it is set to 0.45. We then compute a normalized ratio relative to that target. Properties below 90 percent of the target are labeled undervalued, properties within 90 to 110 percent of the target are labeled fairly valued, and properties above 110 percent of the target are labeled overvalued.

This approach is more policy-grounded than using raw percentile cutoffs because it compares each property's assessment pattern against a benchmark tied to its tax treatment.

In [33]:
```
df["label"].value_counts().plot(kind="bar")
plt.title("Distribution of Valuation Labels")
plt.xlabel("Label")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```



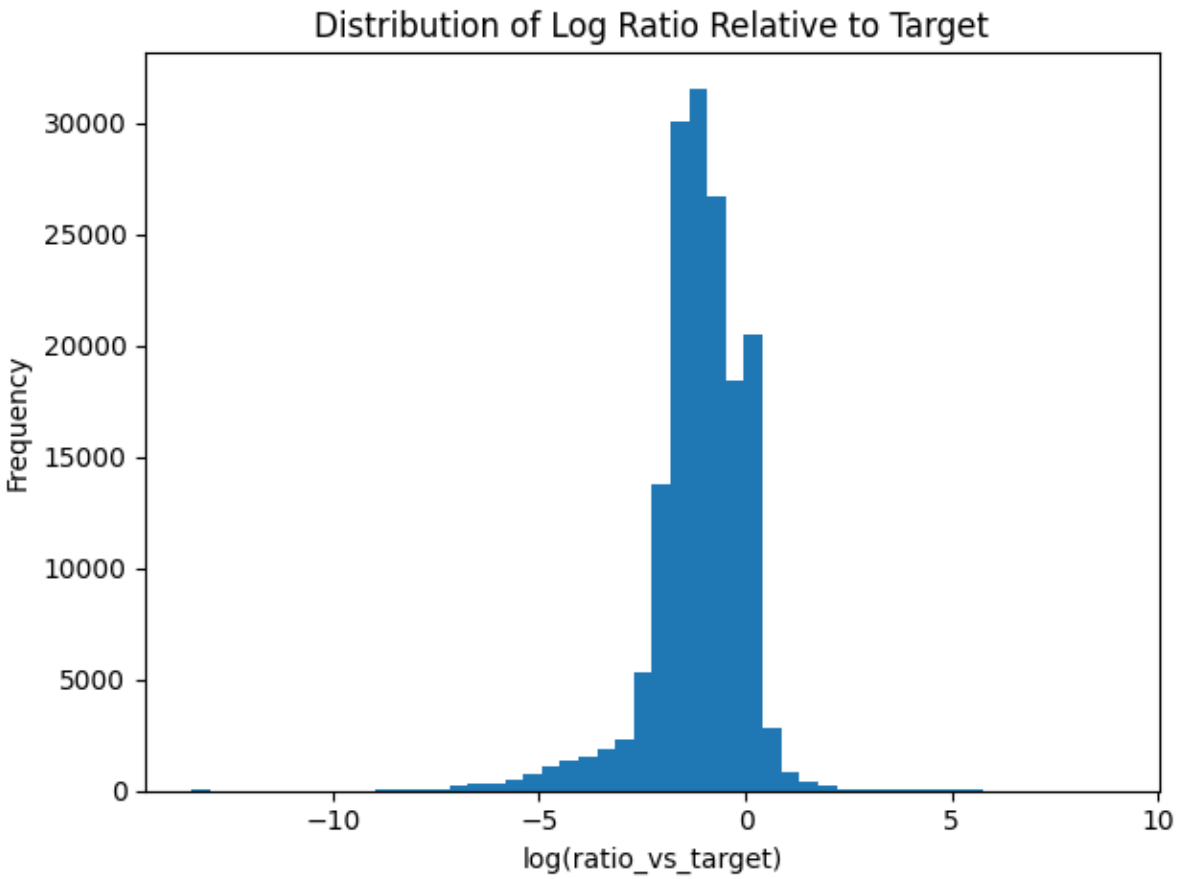

In [34]:
```
df["assessment_ratio"].describe()
```

Out[34]:
```
count    1.610450e+05
mean     1.582266e-01
std      8.700406e+00
min      6.624561e-07
25%      3.967653e-02
50%      6.895385e-02
75%      1.144608e-01
max      3.327830e+03
Name: assessment_ratio, dtype: float64
```

Because the ratio-based variable is highly skewed, we examine a log-transformed version of the ratio relative to the tax-class-specific target. This keeps all observations in the plot while compressing extreme values. A value of zero means a property is exactly at the target ratio, negative values indicate assessment below target, and positive values indicate assessment above target.

In [35]:
```
df["log_ratio_vs_target"] = np.log(df["ratio_vs_target"])

df["log_ratio_vs_target"].dropna().plot(kind="hist", bins=50)
plt.title("Distribution of Log Ratio Relative to Target")
plt.xlabel("log(ratio_vs_target)")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```



Geographic patterns

The dataset covers all five New York City boroughs, although the distribution of observations is uneven. Borough 4 has the largest number of observations, followed by Boroughs 1 and 3, while Boroughs 2 and 5 have smaller representation. This shows that the data provide citywide coverage, but also that sales activity is concentrated more heavily in some boroughs than others.

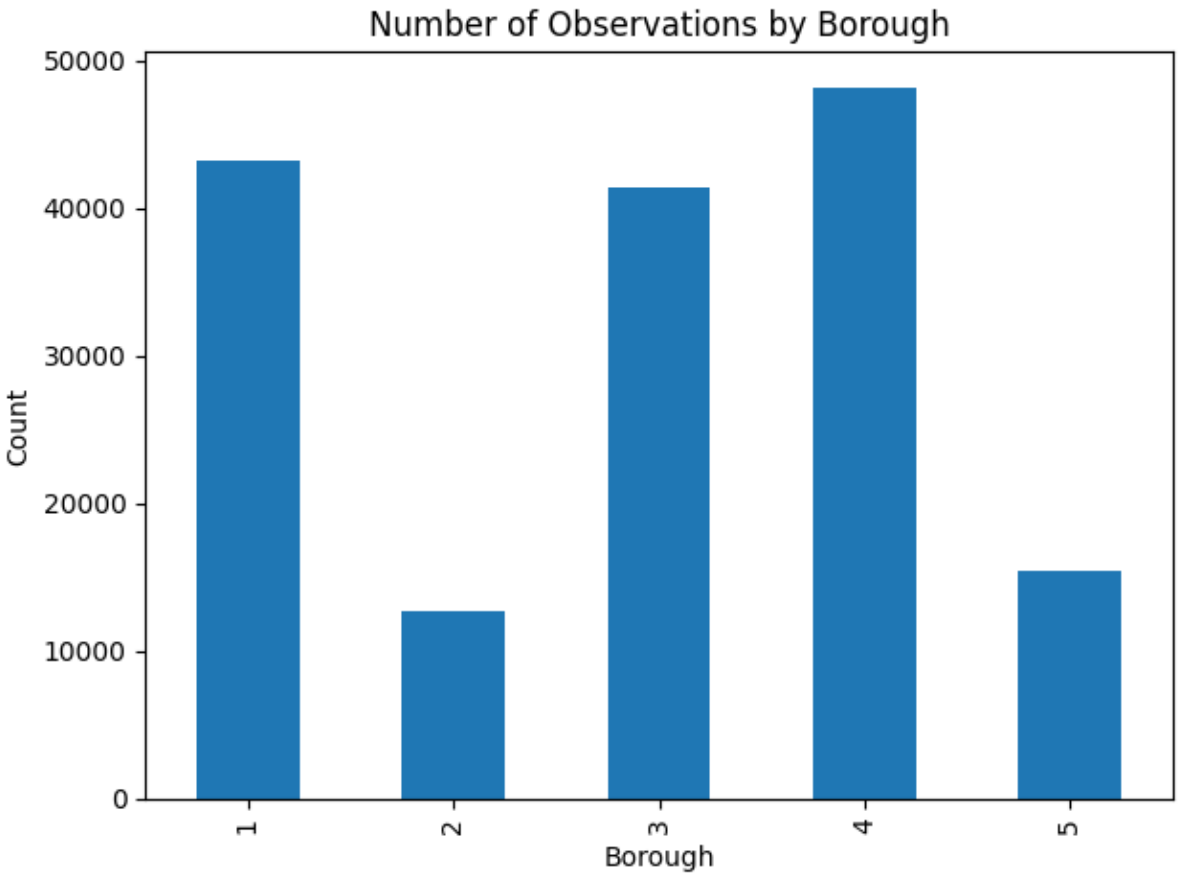
The distribution of valuation labels also varies across boroughs. Some boroughs have larger shares of fairly valued properties, while others show relatively stronger representation of undervalued or overvalued properties. This is one of the most important findings from the exploratory analysis because it suggests that geography plays a meaningful role in valuation outcomes.

The neighborhood counts reinforce this point. The most common neighborhoods in the sample include Flushing-North, Upper East Side, Upper West Side, Midtown East, Chelsea, Bayside, Forest Hills, and Bedford Stuyvesant. These patterns suggest that the sample is concentrated in active and policy-relevant housing markets across the city.

In [36]:
```
df["BOROUGH"].value_counts().sort_index()
```

Out[36]:
```
BOROUGH
1    43294
2    12670
3    41480
4    48195
5    15406
Name: count, dtype: Int64
```

In [37]:
```
df["BOROUGH"].value_counts().sort_index().plot(kind="bar")
plt.title("Number of Observations by Borough")
plt.xlabel("Borough")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

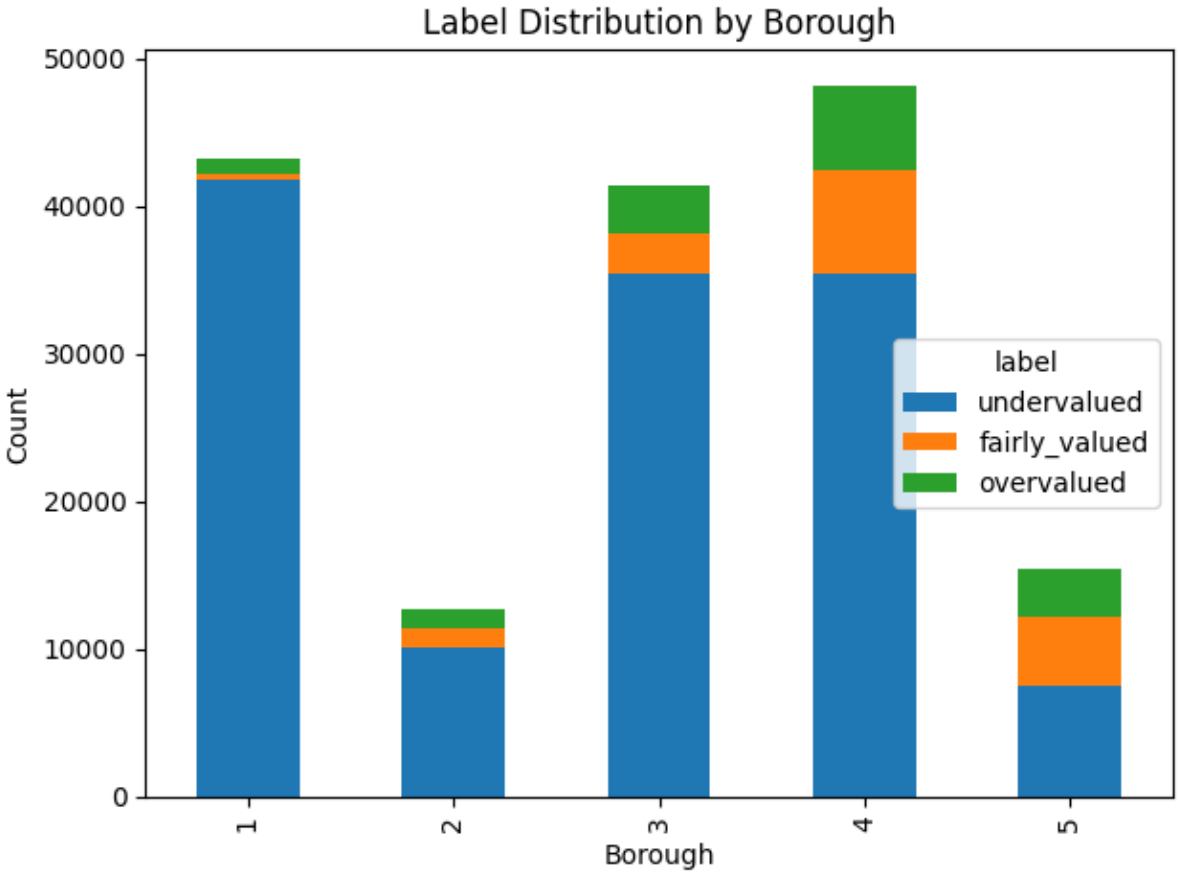



In [38]:
```
borough_label = pd.crosstab(df["BOROUGH"], df["label"])
borough_label
```

Out[38]:

| BOROUGH | label undervalued | fairly_valued | overvalued |
|---|---|---|---|
| **1** | 41801 | 345 | 1148 |
| **2** | 10129 | 1293 | 1248 |
| **3** | 35498 | 2613 | 3369 |
| **4** | 35483 | 6977 | 5735 |
| **5** | 7463 | 4767 | 3176 |

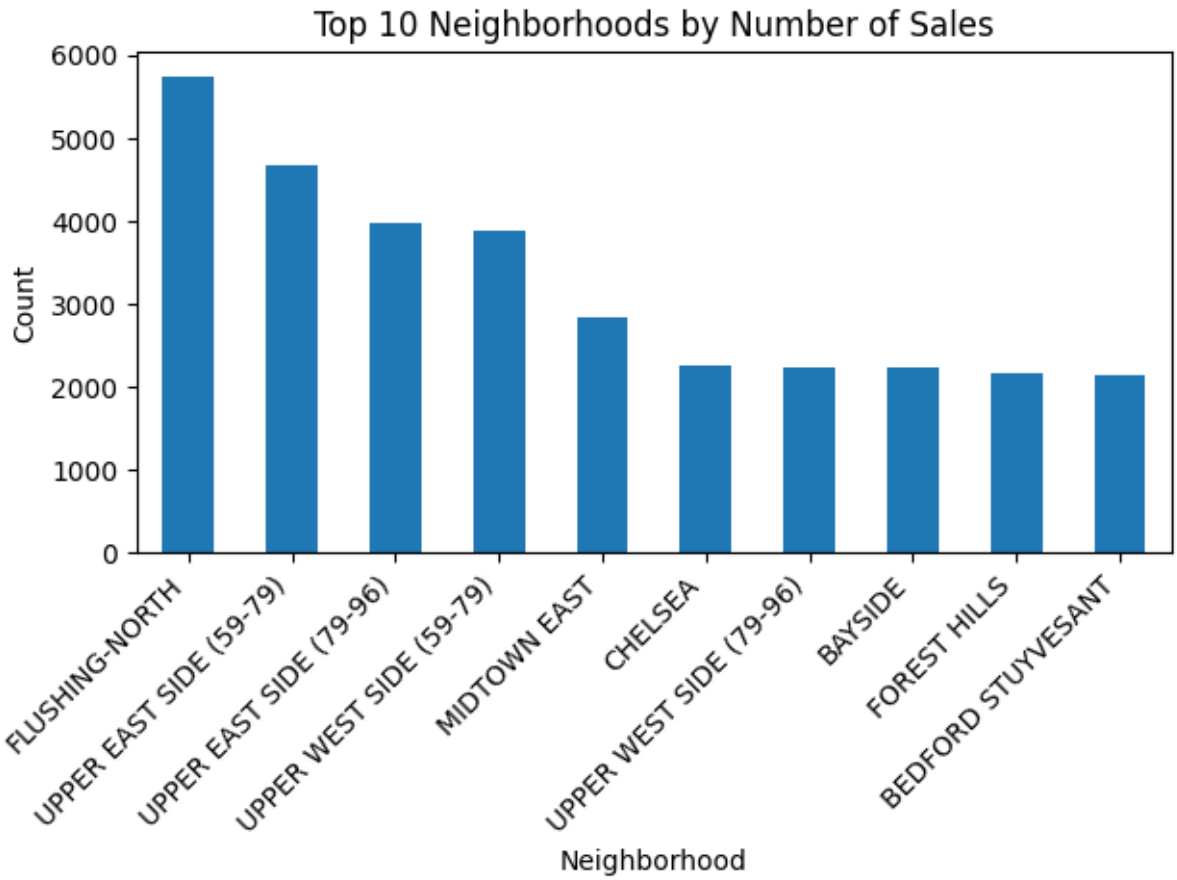
In [39]:
```
borough_label.plot(kind="bar", stacked=True)
plt.title("Label Distribution by Borough")
plt.xlabel("Borough")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```




In [40]:
```
df["NEIGHBORHOOD"].value_counts().head(10)
```

Out[40]:
```
NEIGHBORHOOD
FLUSHING-NORTH              5749
UPPER EAST SIDE (59-79)     4669
UPPER EAST SIDE (79-96)     3982
UPPER WEST SIDE (59-79)     3892
MIDTOWN EAST                2852
CHELSEA                     2267
UPPER WEST SIDE (79-96)     2247
BAYSIDE                     2237
FOREST HILLS                2161
BEDFORD STUYVESANT          2147
Name: count, dtype: int64
```

In [41]:
```
df["NEIGHBORHOOD"].value_counts().head(10).plot(kind="bar")
plt.title("Top 10 Neighborhoods by Number of Sales")
plt.xlabel("Neighborhood")
plt.ylabel("Count")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```




Tax class patterns and overall takeaway

Tax class is another important dimension of the dataset because the target label is defined relative to tax-treatment-based benchmark ratios. Most observations fall within tax classes 1 and 2, while tax class 4 represents a much smaller portion of the sample. After applying the corrected label definition, the distribution of undervalued, fairly valued, and overvalued properties can be compared across tax classes in a way that is more consistent.

The label definition produces a highly imbalanced target distribution. Most properties fall into the undervalued category, while the fairly valued and overvalued classes are much smaller. This reflects the benchmark-based labeling rule used in the modeling pipeline and suggests that class imbalance will need to be considered when evaluating models.

In [42]:
```
tax_label = pd.crosstab(df["TAX CLASS AT TIME OF SALE"], df["label"])
tax_label
```

Out[42]:

| label | undervalued | fairly_valued | overvalued |
|---|---|---|---|
| **TAX CLASS AT TIME OF SALE** | | | |
| **1.0** | 37194 | 15517 | 13221 |
| **2.0** | 83357 | 379 | 1071 |
| **4.0** | 9823 | 99 | 384 |

In [43]:
```
tax_label.plot(kind="bar", stacked=True, figsize=(10,6))
plt.title("Label Distribution by Tax Class")
plt.xlabel("Tax Class at Time of Sale")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```